

AI Knowledge Assistant with RAG using Claude and OpenClaw

Project Guide — v2.0

Project Overview

This project builds an AI-powered document search and question-answering system using Retrieval-Augmented Generation (RAG). It integrates a ChromaDB vector database, a document processing pipeline, and Anthropic's Claude model to provide grounded answers with citations. Documents are uploaded via a Streamlit UI or ingested programmatically, and answers are exposed through a callable OpenClaw agent skill.

Core Architecture

- 1. Document Ingestion:** Load PDFs, extract text, and split into fixed-size chunks.
- 2. Embedding:** Convert text chunks into vector representations using sentence-transformers.
- 3. Vector Storage:** Persist embeddings in ChromaDB for fast similarity search.
- 4. Retrieval:** Given a user query, retrieve the top-k most relevant chunks.
- 5. Generation:** Send retrieved context to Claude API; generate a grounded, cited answer.
- 6. Agent Layer:** OpenClaw manages skill invocation and interaction routing.

Tech Stack

Language: Python 3.10+

LLM: Claude API (Anthropic) — claude-3-5-haiku / claude-3-5-sonnet

Vector Store: ChromaDB

Embeddings: sentence-transformers (local, no API cost)

PDF Parsing: pypdf

Frontend: Streamlit

Agent Framework: OpenClaw

Environment: python-dotenv

System Instructions (Claude Prompt)

The assistant must answer only based on provided retrieved context, avoid hallucination, and always cite sources using the format: (Source: <file_name>, Page <n>). If the context is insufficient, it must explicitly state that the answer cannot be determined.

Project Modules

ingest.py

Loads PDFs from app/data/, extracts text with pypdf, and splits into chunks with metadata (source filename, page number).

retriever.py

Builds a ChromaDB collection from chunks using sentence-transformers embeddings. Exposes a retrieve() function that returns the top-k relevant chunks for a query.

generator.py

Calls the Claude API with the retrieved context and user query. Enforces the citation-based system prompt and returns a formatted answer.

rag_pipeline.py

Orchestrates the full pipeline: calls ingest, builds the vector store, and connects retrieval with generation via a single query() interface.

streamlit_app.py

Streamlit frontend. Sidebar allows users to upload new PDFs and delete indexed ones. Main area accepts a natural-language query and displays Claude's cited answer with expandable chunk view.

openclaw_skill.py

Wraps the RAG pipeline as a callable OpenClaw agent skill. Exposes run(input) accepting a query and optional doc_path. Returns answer, sources, and status.

Project Structure

```
ai-rag-openclaw/  
  app/  
    data/                # PDF documents (default: RAG_Project_Guide.pdf)  
    ingest.py            # PDF loading and chunking  
    retriever.py         # Vector search against ChromaDB  
    generator.py         # Claude API call with retrieved context  
    rag_pipeline.py      # Connects retrieval and generation  
    streamlit_app.py     # Streamlit frontend  
    openclaw_skill.py    # Wraps RAG as a callable OpenClaw skill  
  assets/  
    demo.png             # Demo screenshot  
  .env                   # API keys (not committed)  
  .gitignore  
  requirements.txt  
  README.md
```

Evaluation Strategy

Chunk size: Smaller chunks improve precision; larger chunks preserve context. Test 256, 512, and 1024 token sizes.

Top-k retrieval: Controls how many chunks are passed to Claude. Compare k=3 vs k=5 for accuracy vs. context overload.

Prompt design: Tune the system prompt to reduce hallucination and improve citation accuracy.

Answer accuracy: Run ground-truth Q&A; pairs against ingested documents and measure correctness.

Hallucination rate: Verify that every claim in the answer maps to a retrieved chunk.

Key Concepts

RAG: Combines retrieval and generation to ground LLM outputs in real document content.

Embedding: A vector representation of text that captures semantic meaning.

Chunking: Splitting documents into smaller pieces for more precise retrieval.

Hallucination: When an LLM generates information not supported by the provided context.

Top-k retrieval: Fetching the k most semantically similar chunks to a query.

Resume Description

Built an end-to-end RAG-based AI knowledge assistant integrating Anthropic's Claude and OpenClaw. Implemented PDF ingestion, sentence-transformer embeddings, ChromaDB vector search, and grounded response generation with citations. Delivered a Streamlit UI with live file upload and document management, and packaged the pipeline as a callable OpenClaw agent skill.